MODULE 09 · DOMAIN 2 - DATA PREPARATION

Advanced chunking and re-ranking

Escaping the precision versus context trade-off, and adding a second, sharper look at what retrieval found.

SECTION 2 of 6 EXAM WEIGHT 14% MODULE 09 of 40

admin@sthithapragna.com

Retrieval is a shortlist. Re-ranking is the interview panel. **Search wide and fast, then look closely at only the best few.**

Why simple chunking hits a limit

SMALL CHUNKS

Find the right spot

- Sharp vectors
- Precise matches
- But the answer loses its surroundings
- Conditions a paragraph away go missing

LARGE CHUNKS

Carry the context

- Whole sections together
- Conditions stay attached
- But vectors blur several topics
- Matches become vague

ON THE EXAM

With one chunk doing both jobs, you must pick one. Advanced strategies give each job its own piece of text.

Parent-child chunking: search small, return big

01
Parents
Split into large chunks, about 2,048 tokens.

02
Children
Split each parent into small chunks, about 512.

03
Store
Each child row keeps its parent's text.

04
Search
Embed and match the children.

05
Return
Send the parent text to the LLM.

ON THE EXAM

The child's sharp vector finds the spot. The parent's full text gives the answer its context.

Advanced strategies, and what each one fixes

STRATEGY	HOW IT WORKS	FIXES
Parent-child	Search children, return parent	Precise match, lost context
Sentence window	Embed sentences, return neighbours	The same, at finer grain
Semantic chunking	Cut where meaning shifts	Ideas split by arbitrary cuts
Contextual headers	Prepend title or summary	Chunks meaningless alone
Summaries, questions	Index a summary per chunk	Queries worded unlike text
Hierarchical	Find document, then passage	Long, similar documents

ON THE EXAM

Match the strategy to the symptom. Each exists because one failure kept happening.

Parent-child, or just bigger chunks?

BIGGER CHUNKS

One text, both jobs

- Context improves
- But the vector blurs
- Retrieval gets vaguer
- Fewer fit in the prompt

PARENT-CHILD

Two texts, one job each

- Search stays sharp
- Context is still returned
- More storage per row
- A little more pipeline work

THE TIEBREAKER

When answers lack context but retrieval is precise, do not simply enlarge chunks. Search small and return big.

When documents run to hundreds of pages

- **Hierarchy:** index document and section summaries, find the right section, then its passages.
- **Metadata:** document, section, version and effective date on every chunk.
- **Filters** on that metadata, such as a year or a regulation, at query time.
- **Contextual headers**, so a passage on page 212 still says what it is part of.
- Similar documents from different years look alike to embeddings. Only metadata tells them apart.

> ON THE EXAM
>
> In large collections of similar long documents, the best answer usually combines structure, metadata filters and hierarchy.

Two-stage retrieval

01

Stage 1: recall

Fast vector search pulls a wide set, such as 50.

02

Stage 2: precision

A re-ranker scores each against the question.

03

Keep the best

Only the top few go into the prompt.

ON THE EXAM

Stage 1 decides what can possibly be found. Stage 2 decides what the model actually reads.

Comparing vectors, or reading together?

STAGE 1: BI-ENCODER

Embed separately, compare

- Question and chunk embedded apart
- Chunk vectors computed in advance
- Scales to millions in milliseconds
- Loses fine detail

STAGE 2: CROSS-ENCODER

Read both at once

- Question and chunk read together
- Sees how each word relates
- Far more accurate
- Too slow for more than dozens

ON THE EXAM

The first stage is fast because the chunk vectors were computed earlier. The second is accurate because it reads the pair together.

The AI Search reranker

ASPECT	DETAIL
How	reranker=DatabricksReranker(columns_to_rerank=[...]) in similarity_search
Candidates	Over-fetches 50, then returns num_results
Context	columns_to_rerank adds text and metadata, such as a document summary
Benefit	Typically about 10% better quality
Cost	Adds latency: around 1.5 seconds for 50, faster for short chunks

ON THE EXAM

Databricks recommends trying the reranker for any rag agent. It also offers fine-tuning a reranker on your own data.

Re-rank, or leave it off?

RE-RANK WHEN

Quality first

- A rag agent or chat assistant
- Latency is dominated by LLM generation
- Low to moderate traffic, about 5 QPS as standard
- The right chunk is found but ranked low

LEAVE IT OFF WHEN

Speed first

- A search bar needing under 100 ms
- High traffic, well beyond 5 QPS
- The extra second is unacceptable
- The right chunk is not found at all

THE TIEBREAKER

If latency is the stated priority, the reranker is usually off. If quality is, and an LLM will take seconds anyway, it is usually on.

Which problem, which fix?

SYMPTOM	STAGE	FIX
Right chunk in the top 50, rarely in the top 5	Precision	Re-rank
Right chunk not retrieved at all	Recall	Hybrid search, query expansion, better chunks
Chunk found, but answer lacks its context	Chunking	Parent-child or sentence window
Similar chunks from the wrong year or version	Source	Metadata filters, from module 06

ON THE EXAM

A re-ranker is the right answer only when the right chunk is already somewhere in the candidate set.

Harbourline's retrieval, upgraded

01
Children
Sentences and short clauses, embedded.

›

02
Parents
The full section returned for context.

›

03
Hybrid
Keyword match catches clause numbers.

›

04
Filter
Product and policy year from metadata.

›

05
Re-rank
Top 50 reranked, with section title as context.

ON THE EXAM

A chat assistant, a few requests per second, answers taking seconds anyway. Every signal says the reranker is worth it here.

A rag assistant over insurance policies uses 300-token chunks. Retrieval reliably finds the right paragraph, but answers often miss exclusions stated elsewhere in the same section.

An earlier trial with 3,000-token chunks kept the exclusions, but retrieval became noticeably less accurate.

What should the engineer do?

A Retrieve 50 small chunks per query instead of the current 5

B Move to 4,000-token chunks so that every exclusion is kept

C Search small child chunks, and return their parent section

D Fine-tune the embedding model on a corpus of policy text

◆ PAUSE AND DECIDE

What should the engineer do?

A Fifty small chunks

Floods the prompt with unrelated passages. Exclusions still not attached.

B 4,000-token chunks

Repeats the failed trial, with even blurrier vectors.

C Parent-child

Keeps the precise search and returns the whole section.

D Fine-tune embeddings

Retrieval is already precise. The gap is context, not matching.

THE SIGNAL

Small chunks: precise but no context. Large chunks: context but imprecise. When both have failed, separate search from return.

A chat assistant using AI Search handles about two requests per second. Responses take several seconds, mostly spent generating the answer.

Evaluation shows the correct passage is usually somewhere in the top 40 results, but often not in the top 5 that are sent to the LLM.

Which change will most improve answer quality?

A Increase the chunk size

B Enable the AI Search reranker and keep num_results at 5

C Move the index to a storage-optimised endpoint

D Reduce the overlap between chunks

◆ PAUSE AND DECIDE

Which change will most improve answer quality?

A Bigger chunks

Changes what is stored. The ordering problem remains.

B Reranker, num_results 5

The right passage is already a candidate. Reordering lifts it.

C Storage-optimised endpoint

Changes cost and scale, not ranking quality.

D Less overlap

Fewer records. Nothing to do with order.

THE SIGNAL

In the top 40, not in the top 5: a precision problem. Two requests per second and seconds of generation: the latency is affordable.

> A retailer's product search bar queries an AI Search index as customers type. It serves about 300 queries per second and must respond in under 100 milliseconds.
>
> A team member proposes enabling the reranker to improve result quality.

What is the best response?

A Enable the reranker for every query

B Enable the reranker and have it consider 200 candidates

C Enable the reranker and use larger chunks to compensate

D Keep the reranker off, and improve the first stage instead

◆ PAUSE AND DECIDE

What is the best response?

A Reranker on every query

Adds far more than 100 ms, at 60 times the standard capacity.

B Rerank 200 candidates

Even slower. Makes the latency problem worse.

C Reranker plus larger chunks

Larger chunks rerank more slowly, not faster.

D Off, improve stage one

Meets the latency budget. Tune hybrid search, filters, embeddings.

THE SIGNAL

Under 100 milliseconds and 300 queries per second. Both match the documented cases for leaving the reranker off.

A compliance assistant indexes 20 years of regulatory rulebooks, each around 300 pages. Consecutive editions are largely similar.

Questions such as what did rule 4.2 require in 2019 often return passages from the wrong edition.

What should the engineer do?

A Add edition and effective-date metadata, and filter on it

B Enable the reranker to push the correct edition to the top

C Use much larger chunks so each carries more of its edition

D Increase the overlap between chunks to keep editions apart

◆ PAUSE AND DECIDE

What should the engineer do?

A Metadata and date filter

Only metadata can tell near-identical editions apart.

B Reranker

Scores relevance. Every edition of rule 4.2 is equally relevant.

C Larger chunks

Blurs vectors further. The year is still not in the match.

D More overlap

Affects chunk edges, not which edition is chosen.

THE SIGNAL

Largely similar editions, and a year in the question. When text looks the same, only metadata can separate it.

What to carry forward

- **Parent-child:** search small children, return the parent. Precision and context together.
- Other strategies fix specific symptoms: semantic cuts, headers, summaries, hierarchy.
- Two stages: **recall** finds candidates, **re-ranking** orders them.
- AI Search reranker: fetches 50, about 10% better, adds around a second. Great for rag agents.
- Off for sub-100 ms or high-QPS search. Useless if the right chunk was never retrieved.

> ON THE EXAM
>
> Next: module 10 writes chunks into Delta tables in Unity Catalog, and measures whether retrieval works.